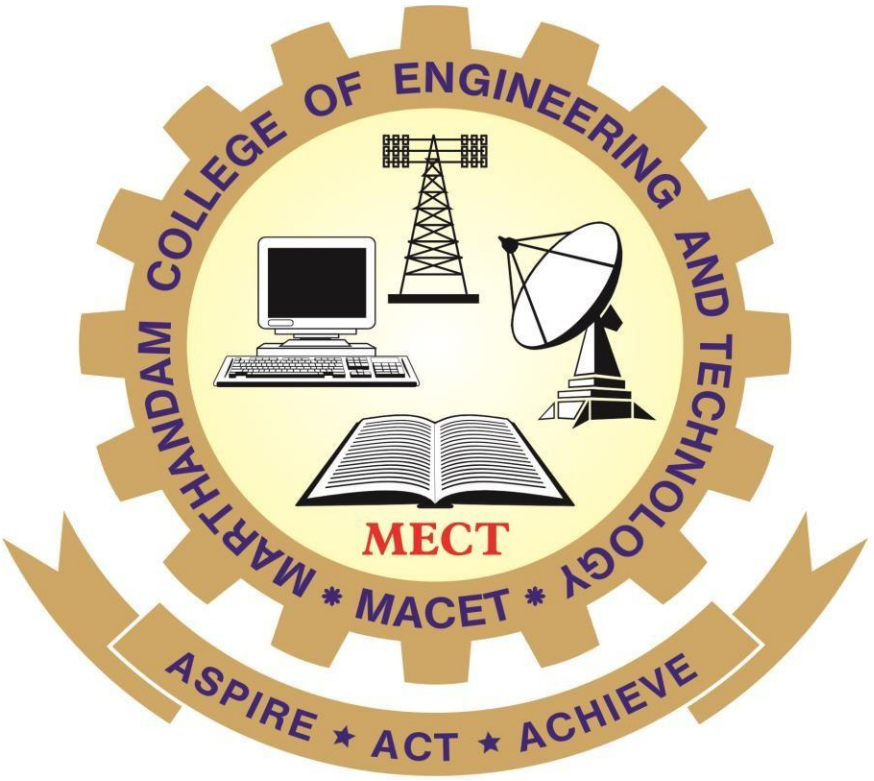
MARTHANDAM COLLEGE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and DataScience



LAB MANUAL

AD3461 MACHINE LEARNING LABORATORY

(IV Semester)

Prepared By

Mrs. W Bivi Evangelin

INDEX

EXP. NO	DATE	NAME OF THE EXPERIMENT	PAGE NO	MARKS	SIGNATURE
1.		For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.			
2.		Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.			
3.		Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.			
4.		Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file and compute the accuracy with a few test data sets.			
5.		Implement naïve Bayesian Classifier model to classify a set of documents and measure the accuracy, precision, and recall.			
6.		Write a program to construct a Bayesian network to diagnose CORONA infection using standard WHO Data Set.			
7.		Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using the k-Means algorithm. Compare the results of these two algorithms.			
8.		Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions.			
9.		Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select an appropriate data set for your experiment and draw graphs.			

Date: **Expt. No.:1**	**For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.**

AIM:

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.

Algorithm

The CANDIDATE-ELIMINTION algorithm computes the version space containing all hypotheses from H that are consistent with an observed sequence of training examples.

Initialize G to the set of maximally general hypotheses in H

Initialize S to the set of maximally specific hypotheses in H

For each training example d, do

- If d is a positive example
- Remove from G any hypothesis inconsistent with d
- For each hypothesis s in S that is not consistent with d
    - Remove s from S
    - Add to S all minimal generalizations h of s such that
        - h is consistent with d, and some member of G is more general than h
    - Remove from S any hypothesis that is more general than another hypothesis in S

- If d is a negative example
    - Remove from S any hypothesis inconsistent with d
    - For each hypothesis g in G that is not consistent with d
        - Remove g from G
        - Add to G all minimal specializations h of g such that
            - h is consistent with d, and some member of S is more specific than h
        - Remove from G any hypothesis that is less general than another hypothesis in G

PROGRAM:

```
import numpy as np
import pandas as pd

data = pd.read_csv('2.csv')
concepts = np.array(data.iloc[:,0:-1])
target = np.array(data.iloc[:,-1])
def learn(concepts, target):
    specific_h = concepts[0].copy()
    print("initialization of specific_h \n",specific_h)
    general_h = [["?" for i in range(len(specific_h))] for i in range(len(specific_h))]
    print("initialization of general_h \n", general_h)

    for i, h in enumerate(concepts):
        if target[i] == "yes":
            print("If instance is Positive ")
            for x in range(len(specific_h)):
                if h[x]!= specific_h[x]:
                    specific_h[x] ='?'
                    general_h[x][x] ='?'

        if target[i] == "no":
            print("If instance is Negative ")
            for x in range(len(specific_h)):
                if h[x]!= specific_h[x]:
                    general_h[x][x] = specific_h[x]
                else:
                    general_h[x][x] = '?'

        print(" step {}".format(i+1))
        print(specific_h)
        print(general_h)
        print("\n")
        print("\n")
```

```
        indices = [i for i, val in enumerate(general_h) if val == ['?', '?', '?', '?', '?', '?']]
        for i in indices:
            general_h.remove(['?', '?', '?', '?', '?', '?'])
        return specific_h, general_h

    s_final, g_final = learn(concepts, target)

    print("Final Specific_h:", s_final, sep="\n")
    print("Final General_h:", g_final, sep="\n")
```

OUTPUT:

```
initialization of specific_h
 ['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
initialization of general_h
 [['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],
 ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

If instance is Positive
 step 1
['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],
['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

If instance is Positive
 step 2
['sunny' 'warm' '?' 'strong' 'warm' 'same']
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],
['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]


If instance is Negative
 step 3
['sunny' 'warm' '?' 'strong' 'warm' 'same']
[['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],
['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'same']]
```

If instance is Positive

step 4

['sunny' 'warm' '?' 'strong' '?' '?']

[['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'],
['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

Final Specific_h:

['sunny' 'warm' '?' 'strong' '?' '?']

Final General_h:

[['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?']]

RESULT

Date: **Expt No:2**	**For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.**

AIM:

To demonstrate the working of the decision tree based ID3 algorithm with an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

ALGORITHM:

- Create a Root node for the tree
- If all Examples are positive, Return the single-node tree Root, with label = +
- If all Examples are negative, Return the single-node tree Root, with label = -
- If Attributes is empty, Return the single-node tree Root, with label = most common value of Target_attribute in Examples

- Otherwise Begin
  - A ← the attribute from Attributes that best* classifies Examples
  - The decision attribute for Root ← A
  - For each possible value, v_i, of A,
    - Add a new tree branch below *Root*, corresponding to the test A = v_i
    - Let *Examples* $_{vi}$, be the subset of Examples that have value v_i for *A*
    - If *Examples* $_{vi}$, is empty
      - Then below this new branch add a leaf node with label = most common value of Target_attribute in Examples
      - Else below this new branch add the subtree
        ID3(*Examples* $_{vi}$, Targe_tattribute, Attributes – {A}))
- End
- Return Root

PROGRAM:

```
import numpy as np
import math
import csv
def read_data(filename):
    with open(filename, 'r') as csvfile:
        datareader = csv.reader(csvfile, delimiter=',')
        headers = next(datareader)
        metadata = []
        traindata = []
        for name in headers:
```

```
            metadata.append(name)
        for row in datareader:
            traindata.append(row)

    return (metadata, traindata)
class Node:
    def __init__(self, attribute):
        self.attribute = attribute
        self.children = []
        self.answer = ""

    def __str__(self):
        return self.attribute
def subtables(data, col, delete):
    dict = {}
    items = np.unique(data[:, col])
    count = np.zeros((items.shape[0], 1), dtype=np.int32)

    for x in range(items.shape[0]):
        for y in range(data.shape[0]):
            if data[y, col] == items[x]:
                count[x] += 1

    for x in range(items.shape[0]):
        dict[items[x]] = np.empty((int(count[x]), data.shape[1]), dtype="|S32")
        pos = 0
        for y in range(data.shape[0]):
            if data[y, col] == items[x]:
                dict[items[x]][pos] = data[y]
                pos += 1
        if delete:
            dict[items[x]] = np.delete(dict[items[x]], col, 1)

    return items, dict
def entropy(S):
    items = np.unique(S)

    if items.size == 1:
        return 0

    counts = np.zeros((items.shape[0], 1))
    sums = 0

    for x in range(items.shape[0]):
        counts[x] = sum(S == items[x]) / (S.size * 1.0)

    for count in counts:
        sums += -1 * count * math.log(count, 2)
    return sums
def gain_ratio(data, col):
    items, dict = subtables(data, col, delete=False)
```

```
    total_size = data.shape[0]
    entropies = np.zeros((items.shape[0], 1))
    intrinsic = np.zeros((items.shape[0], 1))

    for x in range(items.shape[0]):
        ratio = dict[items[x]].shape[0]/(total_size * 1.0)
        entropies[x] = ratio * entropy(dict[items[x]][:, -1])
        intrinsic[x] = ratio * math.log(ratio, 2)

    total_entropy = entropy(data[:, -1])
    iv = -1 * sum(intrinsic)

    for x in range(entropies.shape[0]):
        total_entropy -= entropies[x]

    return total_entropy / iv
def create_node(data, metadata):
    if (np.unique(data[:, -1])).shape[0] == 1:
        node = Node("")
        node.answer = np.unique(data[:, -1])[0]
        return node

    gains = np.zeros((data.shape[1] - 1, 1))

    for col in range(data.shape[1] - 1):
        gains[col] = gain_ratio(data, col)

    split = np.argmax(gains)

    node = Node(metadata[split])
    metadata = np.delete(metadata, split, 0)

    items, dict = subtables(data, split, delete=True)

    for x in range(items.shape[0]):
        child = create_node(dict[items[x]], metadata)
        node.children.append((items[x], child))
    return node
def empty(size):
    s = ""
    for x in range(size):
        s += "  "
    return s

def print_tree(node, level):
    if node.answer != "":
        print(empty(level), node.answer)
        return
    print(empty(level), node.attribute)
    for value, n in node.children:
```

```
        print(empty(level + 1), value)
        print_tree(n, level + 2)
metadata, traindata = read_data("tennisdata.csv")
data = np.array(traindata)
node = create_node(data, metadata)
print_tree(node, 0)
```

OUTPUT:

```
Outlook
   Overcast
      b'Yes'
   Rainy
      Windy
         b'False'
            b'Yes'
         b'True'
            b'No'
   Sunny
      Humidity
         b'High'
            b'No'
         b'Normal'
            b'Yes'
```

RESULT:

Date: Expt No: 3	**Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.**

AIM:

To build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

ALGORITHM:

- Create a feed-forward network with n_i inputs, n_{hidden} hidden units, and n_{out} output units.
- Initialize all network weights to small random numbers
- Until the termination condition is met, Do
  - For each $(\vec{x}, \vec{t})$, in training examples, Do

    Propagate the input forward through the network:

    1. Input the instance $\vec{x}$, to the network and compute the output o_u of every unit u in the network.

    Propagate the errors backward through the network:

    2. For each network output unit k, calculate its error term δ_k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k)$$

    3. For each hidden unit *h*, calculate its error term δ_h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in outputs} w_{h,k}\delta_k$$

    4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = \eta \delta_j x_{i,j}$$

PROGRAM:

```
import numpy as np

X = np.array(([2, 9], [1, 5], [3, 6]), dtype=float)
y = np.array(([92], [86], [89]), dtype=float)
X = X/np.amax(X,axis=0) #maximum of X array longitudinally
y = y/100
def sigmoid (x):
    return 1/(1 + np.exp(-x))
def derivatives_sigmoid(x):
    return x * (1 - x)
epoch=5
lr=0.1
inputlayer_neurons = 2
hiddenlayer_neurons = 3
output_neurons = 1
wh=np.random.uniform(size=(inputlayer_neurons,hiddenlayer_neurons))
bh=np.random.uniform(size=(1,hiddenlayer_neurons))
wout=np.random.uniform(size=(hiddenlayer_neurons,output_neurons))
bout=np.random.uniform(size=(1,output_neurons))
EO = y-output
outgrad = derivatives_sigmoid(output)
d_output = EO * outgrad
EH = d_output.dot(wout.T)
hiddengrad = derivatives_sigmoid(hlayer_act)
d_hiddenlayer = EH * hiddengrad
wout += hlayer_act.T.dot(d_output) *lr
wh += X.T.dot(d_hiddenlayer) *lr
print ("-----------Epoch-", i+1, "Starts----------")
print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n" ,output)
print ("-----------Epoch-", i+1, "Ends---------- \n")
```

OUTPUT:

Input:

```
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
```

Actual Output:

```
[[0.92]
 [0.86]
 [0.89]]
```

Predicted Output:

[[0.86039886]

[0.8461084]

[0.86187001]]

RESULT:

Date: **Expt No : 4**	**Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.**

AIM:

To write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file and compute the accuracy of the classifier.

Naïve Bayesian Classifier Algorithm

Step 1: Convert the data set into a frequency table

Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

PROGRAM

```
import csv
import random
import math

def safe_div(x,y):
    if y == 0:
        return 0
    return x/y
```

```
def loadcsv(filename):
    lines = csv.reader(open(filename, "r"));
    dataset = list(lines)
    for i in range(len(dataset)):
        dataset[i] = [float(x) for x in dataset[i]]
    return dataset

def splitdataset(dataset, splitratio):
    trainsize = int(len(dataset) * splitratio);
    trainset = []
    copy = list(dataset);
    while len(trainset) < trainsize:
        index = random.randrange(len(copy));
        trainset.append(copy.pop(index))
    return [trainset, copy]

def separatebyclass(dataset):
    separated = {}
    for i in range(len(dataset)):
        vector = dataset[i]
        if (vector[-1] not in separated):
            separated[vector[-1]] = []
        separated[vector[-1]].append(vector)
    return separated

def mean(numbers):
    return sum(numbers)/float(len(numbers))

def stdev(numbers):
    avg = mean(numbers)
    variance = sum([pow(x-avg,2) for x in
numbers])/float(len(numbers)-1)
    return math.sqrt(variance)

def summarize(dataset):
    summaries = [(mean(attribute), stdev(attribute)) for
attribute in zip(*dataset)];
    del summaries[-1]
    return summaries

def summarizebyclass(dataset):
    separated = separatebyclass(dataset);
    summaries = {}
    for classvalue, instances in separated.items():
        summaries[classvalue] = summarize(instances)
    return summaries

def calculateprobability(x, mean, stdev):
    exponent = math.exp(-(math.pow(x-mean,2)/
(2*math.pow(stdev,2))))
```

```
 return (1 / (math.sqrt(2*math.pi) * stdev)) * exponent

def calculateclassprobabilities(summaries, inputvector):
    probabilities = {}
    for classvalue, classsummaries in summaries.items():
        probabilities[classvalue] = 1
        for i in range(len(classsummaries)):
            mean, stdev = classsummaries[i]
            x = inputvector[i]
            calculateprobability(x, mean, stdev);
    return probabilities
def predict(summaries, inputvector):
    probabilities = calculateclassprobabilities(summaries,
inputvector)
    bestLabel, bestProb = None, -1
    for classvalue, probability in probabilities.items():
        if bestLabel is None or probability > bestProb:
            bestProb = probability
            bestLabel = classvalue
    return bestLabel

def getpredictions(summaries, testset):
    predictions = []
    for i in range(len(testset)):
        result = predict(summaries, testset[i])
        predictions.append(result)
    return predictions

def getaccuracy(testset, predictions):
    correct = 0
    for i in range(len(testSet)):
        if testSet[i][-1] == predictions[i]:
            correct += 1
        return (correct/float(len(testset))) * 100.0

def main():
    filename='C:\python\prima.csv'
    splitratio = 0.67
    dataset = loadcsv(filename);
    trainingSet, testset = splitDataset(dataset, splitratio)
    print('Split {0} rows into train={1} and test={2} rows'.format(len(dataset), len(trainingset), len(testset)))
    summaries = summarizeByClass(trainingSet);
    predictions = getPredictions(summaries, testSet)
    accuracy = getaccuracy(testSet, predictions)
    print('Accuracy of the classifier is :{0}%'.format(accuracy))

main()
```

Output:

The length of the Data Set : 768

The Data Set Splitting into Training and Testing

Number of Rows in Training Set:514 rows

Number of Rows in Testing Set:254 rows

First Five Rows of Training Set:

[4.0, 116.0, 72.0, 12.0, 87.0, 22.1, 0.463, 37.0, 0.0]

[0.0, 84.0, 64.0, 22.0, 66.0, 35.8, 0.545, 21.0, 0.0]

[0.0, 162.0, 76.0, 36.0, 0.0, 49.6, 0.364, 26.0, 1.0]

[10.0, 101.0, 86.0, 37.0, 0.0, 45.6, 1.136, 38.0, 1.0]

[5.0, 78.0, 48.0, 0.0, 0.0, 33.7, 0.654, 25.0, 0.0]

First Five Rows of Testing Set:

[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0, 0.0]

[8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0, 1.0]

[4.0, 110.0, 92.0, 0.0, 0.0, 37.6, 0.191, 30.0, 0.0]

[10.0, 139.0, 80.0, 0.0, 0.0, 27.1, 1.441, 57.0, 0.0]

[7.0, 100.0, 0.0, 0.0, 0.0, 30.0, 0.484, 32.0, 1.0]

Model Summaries:

{0.0: [(3.3474320241691844, 3.045635385378286), (111.54380664652568, 26.040069054720693), (68.45921450151057, 18.15540652389224), (19.94561933534743, 14.709615608767137), (71.50151057401813, 101.04863439385403), (30.863141993957708, 7.207208162103949), (0.4341842900302116,

0.2960911906946818), (31.613293051359516, 12.100651311117689)], 1.0: [(4.469945355191257, 3.7369440851983082), (139.3879781420765, 33.733070931373234), (71.14754098360656, 20.694403393963842), (22.92896174863388, 18.151995092528765), (107.97814207650273, 146.92526156736633), (35.28633879781422, 7.783342260348583), (0.5569726775956286, 0.3942245334398509), (36.78688524590164, 11.174610282702282)]}

Predictions:

[0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0]

RESULT:

Date: Expt No: 5	**Implement naïve Bayesian Classifier model to classify a set of documents and measure the accuracy, precision, and recall.**

AIM:

To implement naïve Bayesian Classifier model to classify a set of documents and measure the accuracy, precision, and recall.

ALGORITHM:

1. *collect all words, punctuation, and other tokens that occur in Examples*
   - *Vocabulary* ← ***c*** the set of all distinct words and other tokens occurring in any text document from *Examples*

2. *calculate the required $P(v_j)$ and $P(w_k|v_j)$ probability terms*

   - For each target value v_j in V do
     - $docs_j$ ← the subset of documents from *Examples* for which the target value is *vj*
     - $P(v_j)$ ← $|\ docs_j\ | / |\text{Examples}|$
     - $Text_j$ ← a single document created by concatenating all members of $docs_j$
     - n ← total number of distinct word positions in $Text_j$
     - for each word w_k in *Vocabulary*
       - n_k ← number of times word $\boldsymbol{w_k}$ occurs in $Text_j$
       - $P(w_k|v_j)$ ← $(n_k + 1) / (n + |\ Vocabulary|\ )$

CLASSIFY_NAIVE_BAYES_TEXT (Doc)

Return the estimated target value for the document Doc. a_i denotes the word found in the i^{th} position within Doc.

- *positions* ← all word positions in *Doc* that contain tokens found in *Vocabulary*
- Return V_{NB}, where

$$v_{NB} = \underset{v_j \in V}{\text{argmax}}\ P(v_j) \prod_{i \in positions} P(a_i|v_j)$$

PROGRAM:

```
import pandas as pd
msg=pd.read_csv('C:/python/naivetext.csv',names=['message','label'])
print('The dimensions of the dataset',msg.shape)
msg['labelnum']=msg.label.map({'pos':1,'neg':0})
X=msg.message
y=msg.labelnum
print(X)
print(y)

from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(X,y)
print ('\n The total number of Training Data :',ytrain.shape)
print ('\n The total number of Test Data :',ytest.shape)

from sklearn.feature_extraction.text import CountVectorizer
count_vect = CountVectorizer()
xtrain_dtm = count_vect.fit_transform(xtrain)
xtest_dtm=count_vect.transform(xtest)
print('\n The words or Tokens in the text documents \n')
print(count_vect.get_feature_names())
df=pd.DataFrame(xtrain_dtm.toarray(),columns=count_vect.get_feature_names())

from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB().fit(xtrain_dtm,ytrain)
predicted = clf.predict(xtest_dtm)

from sklearn import metrics
print('\n Accuracy of the classifer is',
    metrics.accuracy_score(ytest,predicted))

print('\n Confusion matrix')
print(metrics.confusion_matrix(ytest,predicted))
print('\n The value of Precision' ,
metrics.precision_score(ytest,predicted))
print('\n The value of Recall' ,
metrics.recall_score(ytest,predicted))
```

OUTPUT:

```
The dimensions of the dataset (18, 2)
0                    I love this sandwich
1               This is an amazing place
2       I feel very good about these beers
3                    This is my best work
4                    What an awesome view
5           I do not like this restaurant
6                I am tired of this stuff
7                  I can't deal with this
8                    He is my sworn enemy
```

```
9                    My boss is horrible
10                 This is an awesome place
11    I do not like the taste of this juice
12                         I love to dance
13        I am sick and tired of this place
14                   What a great holiday
15           That is a bad locality to stay
16           We will have good fun tomorrow
17         I went to my enemy's house today
Name: message, dtype: object
0     1
1     1
2     1
3     1
4     1
5     0
6     0
7     0
8     0
9     0
10    1
11    0
12    1
13    0
14    1
15    0
16    1
17    0
The total number of Training Data : (13,)

 The total number of Test Data : (5,)



Accuracy of the classifer is 0.8


Confusion matrix
[[3 0]
 [1 1]]

 The value of Precision 1.0

 The value of Recall 0.5
```

Date: Expt No: 6	**Write a program to construct a Bayesian network to diagnose CORONA infection using standard WHO Data set**

AIM:

To write a program to construct a Bayesian network to diagnose CORONA infection using standard WHO Data set.

Data set:

Attribute Information:

1. Breathing Problem
2. Fever
3. Dry cough
4. Sore throat
5. Running Nose
6. Asthma
7. Chronic Lung disease
8. Headache
9. Heart disease
10. Diabetes
11. Hyper tension

PROGRAM:

```
import pandas as pd
import numpy as np
import time
import json

def pre_processing(df):
    print("Processing Test Data...")
    time.sleep(2)
    x = df.drop(df.columns[-1],axis=1)
    y = df[df.columns[-1]]
    print("Done!\n")
    return x, y

def test_model(df,x,y,testData,trained_data):
    print(f"Model Testing In Progress...")
```

```
    time.sleep(3)
    total=len(df[y.name])
    totalYes=trained_data["totalYes"]
    totalNo=trained_data["totalNo"]
    sum1Yes=(totalYes/total)
    sum2Yes=1
    for indx2 in range(len(testData)):
        sum1Yes*=(trained_data[x.columns[indx2]][testData[indx2]]['yes']/totalYes)
        sum2Yes*=(trained_data[x.columns[indx2]][testData[indx2]]['total']/total)
    sumYes = sum1Yes/sum2Yes
    sum1No=(totalNo/total)
    sum2No=1
    for indx2 in range(len(testData)):
        sum1No*=(trained_data[x.columns[indx2]][testData[indx2]]['no']/totalNo)
        sum2No*=(trained_data[x.columns[indx2]][testData[indx2]]['total']/total)
    sumNo = sum1No/sum2No
    print(f"\nProbability of Covid 19 to be True : {format(sumYes,'.2f')}")
    print(f"Probability of Covid 19 to be False : {format(sumNo,'.2f')}")
    if sumYes>sumNo:
        ans="Yes"
    else:
        ans="No"
    print(f"Answer is - {ans}\n\n\n")

def read_data():
    print("Reading data from csv...")
    time.sleep(2)
    df = pd.read_csv("C:/python/covid.csv")
    print("Done...\n")
    return df

def load_model():
    print("Loading Trained Data from trained_data.txt...")
    time.sleep(2)
    with open('C:/python/trained_data.txt') as f:
        data = f.read()
        js = json.loads(data)
    print("Loaded!\n")
    return js

df = pd.DataFrame(read_data())
x,y=pre_processing(df)
x=x.truncate(0,4433)
y=y.truncate(0,4433)
trained_data=load_model()
while True:
    print("\nEnter you choice of features:\n")
    test=[]
    for uniqueCol in list(x.columns):
        columnData=list(np.unique(df[uniqueCol]))
        for idx in range(len(columnData)):
```

```
            print(f"{idx}. {columnData[idx]}")
        inp=int(input(f"Enter option for {uniqueCol}: "))
        test.append(columnData[inp])
        print("\n")
    print(f"Your test data is: {test}\n")
    test_model(df,x,y,test,trained_data)
```

OUTPUT:

```
Reading data from csv...
Done...

Processing Test Data...
Done!

Loading Trained Data from trained_data.txt...
Loaded!


Enter you choice of features:

0. No
1. Yes
Enter option for Breathing Problem: 1


0. No
1. Yes
Enter option for Fever: 1


0. No
1. Yes
Enter option for Dry Cough: 1


0. No
1. Yes
Enter option for Sore throat: 1


0. No
1. Yes
Enter option for Running Nose: 0


0. No
1. Yes
Enter option for Asthma: 0
```

0. No
1. Yes
Enter option for Chronic Lung Disease: 0

0. No
1. Yes
Enter option for Headache: 1

0. No
1. Yes
Enter option for Heart Disease: 0

0. No
1. Yes
Enter option for Diabetes: 0

0. No
1. Yes
Enter option for Hyper Tension: 0

0. No
1. Yes
Enter option for Abroad travel: 1

0. No
1. Yes
Enter option for Contact with COVID Patient: 0

0. No
1. Yes
Enter option for Attended Large Gathering: 1

0. No
1. Yes
Enter option for Visited Public Exposed Places: 0

0. No
1. Yes
Enter option for Family working in Public Exposed Places: 1

0. No
1. Yes
Enter option for Wearing Masks: 1

0. No
1. Yes
Enter option for Sanitization from Market: 1

Your test data is: ['Yes', 'Yes', 'Yes', 'Yes', 'No', 'No', 'No', 'Yes', 'No', 'No', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes']

Model Testing In Progress...

Probability of Covid 19 to be True : 0.92
Probability of Covid 19 to be False : 0.00
Answer is - Yes

RESULT:

Date: **Expt No: 7**	**Apply EM ALGORITHM to classify a set of data stored in a .CSV file. Use the same data set for clustering using the k-means algorithm. Compare the results of these two algorithms.**

AIM:

To write a program for EM ALGORITHM to classify a set of data stored in a .CSV file and use the same data set for clustering using the k-means algorithm**.**

EM ALGORITHM

These are the two basic steps of the EM algorithm, namely **E Step or Expectation Step or Estimation Step** and **M Step or Maximization Step**.

- **Estimation step:**
  - initialize μ_k,$\sum k$ and $\pi_k$by some random values, or by K means clustering results or by hierarchical clustering results.
  - Then for those given parameter values, estimate the value of the latent variables (i.e γ_k)
- **Maximization Step:**
  - Update the value of the parameters(i.e. μ_k,$\sum k$ and π_k) calculated using ML method.

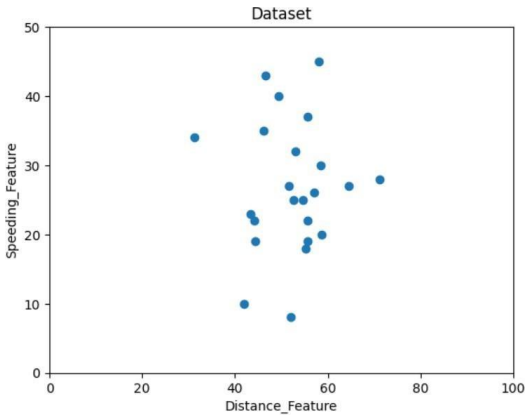
1. Load data set
2. Initialize the mean μ_k, the covariance matrix $\sum k$ and the mixing coefficients
    1. $\pi_k$by some random values. (or other values)
3. Compute the γ_k values for all k.
4. Again Estimate all the parameters using the current γ_k values.
5. Compute log-likelihood function.
6. Put some convergence criterion
7. If the log-likelihood value converges to some value (or if all the parameters converge to some values) then **stop**, else return to **Step 3**.

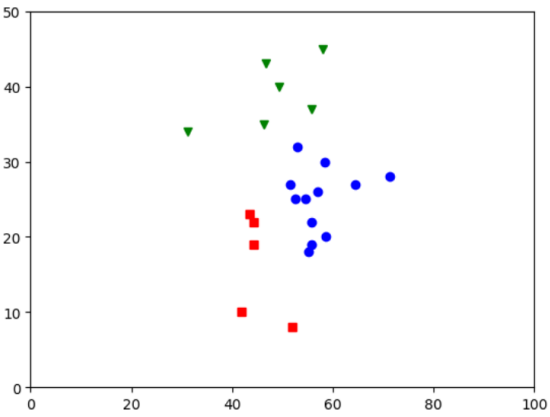
PROGRAM

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt


data = pd.read_csv('C:\python\kmeansdata.csv')
df = pd.DataFrame(data)
print(df)
```

```
col1 = df['Distance_Feature']
col2 = df['Speeding_Feature']
print(list(zip(col1,col2)))


X = np.matrix(list(zip(col1,col2)))


plt.xlim([0, 100])
plt.ylim([0, 50])
plt.title('Dataset')
plt.ylabel('Speeding_Feature')
plt.xlabel('Distance_Feature')


plt.scatter(col1,col2)
plt.plot()
plt.show()


colors = ['b','g','r']
markers = ['o','v','s']


kmeans_model = KMeans(n_clusters=3).fit(X)
plt.plot()


for i,l in enumerate(kmeans_model.labels_):
    plt.plot(col1[i], col2[i], color=colors[l], marker=markers[l], ls='None')
    plt.xlim([0, 100])
    plt.ylim([0, 50])


plt.show()
```

OUTPUT





RESULT:

Date: Expt No: 8	**Write a program to implement k- Nearest Neighbour algorithm to classify the iris data set.**

AIM:

To write a program to implement k- Nearest Neighbour algorithm to classify the iris data set.

K-Nearest Neighbour Algorithm

Training algorithm:

- For each training example (x, f (x)), add the example to the list training examples

Classification algorithm:

- Given a query instance x_q to be classified,
  - Let $x_1 \ldots x_k$ denote the k instances from training examples that are nearest to x_q
  - Return

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} f(x_i)}{k}$$

  - Where, $f(x_i)$ function to calculate the mean value of the k nearest training examples.

Data Set:

Iris Plants Dataset: Dataset contains 150 instances (50 in each of three classes)
Number of Attributes: 4 numeric, predictive attributes and the Class

	sepal-length	sepal-width	petal-length	petal-width	Class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

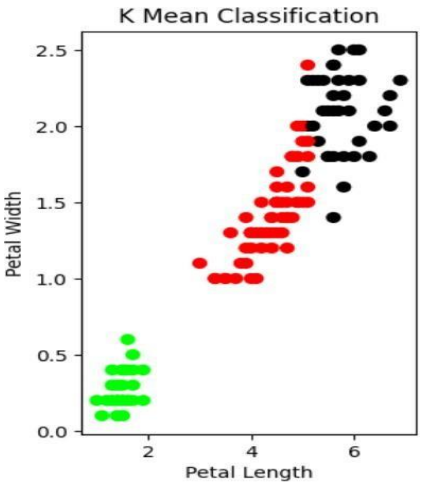
PROGRAM:

```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans
import sklearn.metrics as sm
import pandas as pd
import numpy as np
```

```
iris = datasets.load_iris()


X = pd.DataFrame(iris.data)
X.columns = ['Sepal_Length','Sepal_Width','Petal_Length','Petal_Width']


y = pd.DataFrame(iris.target)
y.columns = ['Targets']


model = KMeans(n_clusters=3)
model.fit(X)


plt.figure(figsize=(14,7))
colormap = np.array(['red', 'lime', 'black'])


plt.subplot(1, 2, 1)
plt.scatter(X.Petal_Length, X.Petal_Width, c=colormap[y.Targets], s=40)
plt.title('Real Classification')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')


from sklearn import preprocessing
scaler = preprocessing.StandardScaler()
scaler.fit(X)
xsa = scaler.transform(X)
xs = pd.DataFrame(xsa, columns = X.columns)
```

OUTPUT:

The accuracy score of K-Mean: 0.24
The Confusion matrixof K-Mean: [[0 50 0]
 [48 0 2]
 [14 0 36]]



RESULT:

Date: **Expt No: 9**	**Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs**.

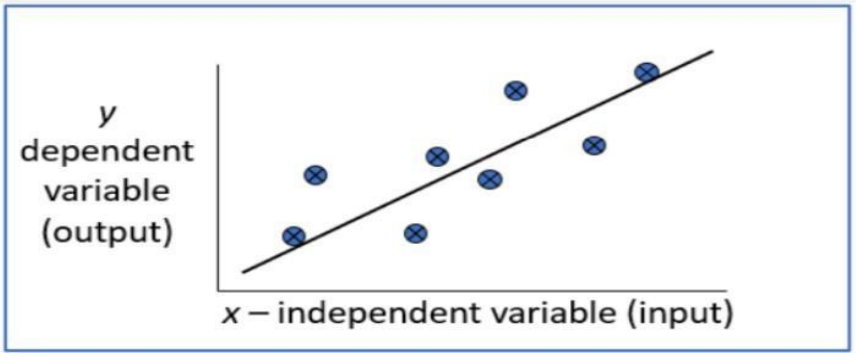
AIM:

To implement the non-parametric Locally Weighted Regression algorithm in order to fit data points and select appropriate data set for your experiment and draw graphs.

Locally Weighted Regression Algorithm

Regression:

- Regression is a technique from statistics that is used to predict values of a desired target quantity when the target quantity is continuous.
- In regression, we seek to identify (or estimate) a continuous variable y associated with a given input vector x.
  - y is called the dependent variable.
  - x is called the independent variable.



Loess/Lowess Regression:

Loess regression is a nonparametric technique that uses local weighted regression to fit a smooth curve through points in a scatter plot.

Lowess Algorithm:

- Locally weighted regression is a very powerful nonparametric model used in statistical learning.
- Given a dataset X, y, we attempt to find a model parameter β(x) that minimizes residual sum of weighted squared errors.
- The weights are given by a kernel function (k or w) which can be chosen arbitrarily

ALGORITHM

1. Read the Given data Sample to X and the curve (linear or non linear) to Y
2. Set the value for Smoothening parameter or Free parameter say τ
3. Set the bias /Point of interest set x0 which is a subset of X
4. Determine the weight matrix using :

$$w(x, x_o) = e^{-\frac{(x-x_o)^2}{2\tau^2}}$$

5. Determine the value of model term parameter β using :

$$\hat{\beta}(x_o) = (X^T W X)^{-1} X^T W y$$

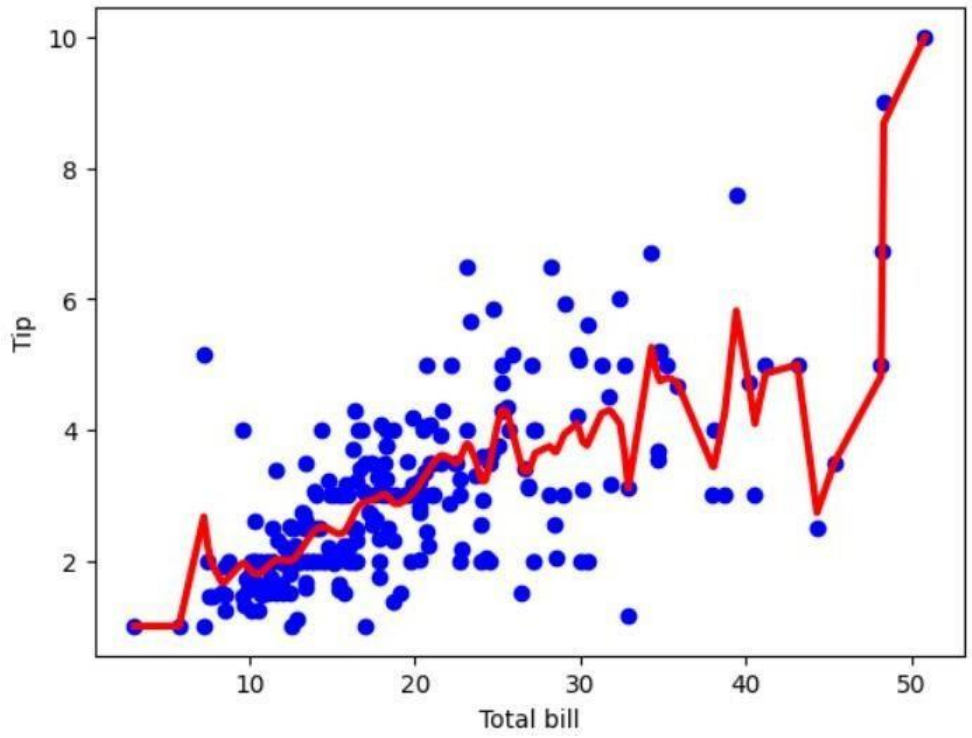
6. Prediction = x0*β:

PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
df = pd.read_csv('C:/python/tips.csv')
features = np.array(df.total_bill)
labels = np.array(df.tip)


def kernel(data, point, xmat, k):
  m,n = np.shape(xmat)
  ws = np.mat(np.eye((m)))
  for j in range(m):
    diff = point - data[j]
    ws[j,j] = np.exp(diff*diff.T/(-2.0*k**2))
  return ws


def local_weight(data, point, xmat, ymat, k):
  wei = kernel(data, point, xmat, k)
  return (data.T*(wei*data)).I*(data.T*(wei*ymat.T))

def local_weight_regression(xmat, ymat, k):
```

```
m,n = np.shape(xmat)
ypred = np.zeros(m)
for i in range(m):
   ypred[i] = xmat[i]*local_weight(xmat, xmat[i],xmat,ymat,k)
return ypred

m = features.shape[0]
mtip = np.mat(labels)
data = np.hstack((np.ones((m, 1)), np.mat(features).T))
ypred = local_weight_regression(data, mtip, 0.5)
indices = data[:,1].argsort(0)
xsort = data[indices][:,0]
 fig = plt.figure()
 ax = fig.add_subplot(1,1,1)
 ax.scatter(features, labels, color='blue')
 ax.plot(xsort[:,1],ypred[indices], color = 'red', linewidth=3)
 plt.xlabel('Total bill')
 plt.ylabel('Tip')
 plt.show()
```

OUTPUT:



RESULT: